

NovaCart: 11-Speaker Presentation Script & Stage Defense

A Story-Driven Architectural Walkthrough & Viva Defense Suite for the DBMS Innovative Examination

Target Event: DBMS Innovative Examination (CS301 / IE)	Team Composition: 11 Presenters (~1.5–2 min / speaker)
Core Focus: Conceptual EER Modeling, Disjoint (\$d\$) & Relational Mapping	Interactive Demo: Live Streamlit Prototype & SQLite Engine

1. Presentation Narrative & Story Arc

To maximize academic evaluation marks, the 11 teammates present NovaCart not as disjointed technical points, but as an **unbroken narrative**: from a real-world multi-vendor e-commerce crisis, to formal Chen EER modeling, to relational mapping mechanics, and finally to live software execution and transaction verification.

#	Speaker & Role	Primary Screen / Tab	Core DBMS Academic Concept
S1	Problem Architect	Title Banner & Header	Multi-vendor personas, Flat-table flaws, Sparse NULLs
S2	EER Conceptual Modeler	Tab 1: Canvas Top (Users & d)	Superclass Entity, Disjoint Specialization Circle (d)
S3	Cardinality Specialist	Tab 1: Canvas Middle (Subclasses)	Customers/Sellers, 1:N 'places' & 'offers', Participation
S4	Decomposition Specialist	Tab 1: Canvas Bottom (Bridge)	M:N Decomposition, Associative Entity Order_Items
S5	Live Canvas Navigator	Tab 1: Element Inspector	Entity metadata classification, Live SQLite tuples
S6	Relational Schema Architect	Tab 2: Inheritance Rules	Subclass PK=FK inheritance, ON DELETE CASCADE, Rules 1–4
S7	Inheritance Demonstrator	Tab 2: Form & Live Join	Atomic two-tier insertion, Natural EER equi-join query
S8	Marketplace Demonstrator	Tab 3: Marketplace Config	Real-time order composition, Vendor catalog interaction
S9	Engine & ACID Specialist	Tab 3: Timeline & Catalog	BEGIN, Orders, Order_Items, CHECK(stock >= 0), COMMIT
S10	Query & Join Specialist	Tab 4: Multi-Table Graph	6-Table join traversal pipeline, Lossless reconstruction
S11	Lead Defender & Wrap-up	Tab 5: Schema Inspector	ON DELETE RESTRICT vs CASCADE, Formal viva conclusion

2. Detailed Speaker Cards & Spoken Scripts

Speaker 1: The Hook & The Multi-Vendor Dilemma	Screen: Main App Title Banner & Header Status Badges Time: 1.5 mins
Spoken Script: <i>"Good morning, respected professors and evaluators. Today, our team is thrilled to present NovaCart, our database engineering prototype built for this DBMS Innovative Examination.</i> <i>Let's begin with a story that every real-world e-commerce giant faces: In any marketplace, you have two fundamentally different personas: Buyers, who have shipping addresses and loyalty tiers, and Merchants, who hold tax registrations (GSTIN), store names, and ratings.</i> <i>What would a naive design do? It would dump everyone into a single flat table called Users. But that creates a disaster: for every buyer, merchant columns are empty—50% sparse NULL values across the entire database! Furthermore, storing order lines directly inside that table duplicates seller details across thousands of rows, and deleting an order risks accidentally purging customer accounts.</i> <i>To solve this cleanly, we designed NovaCart using Enhanced Entity-Relationship (EER) Modeling. I'll now pass the floor to Speaker 2, who will show you the conceptual blueprint that solves this dilemma."</i>	
DBMS Keywords to Emphasize: Multi-vendor personas, Flat-table defects, Sparse NULL proliferation, Conceptual EER architecture.	Handover Segue: "Over to Speaker 2 to unveil our Chen EER Architecture Canvas."
Examiner Question: "Why not just create two separate tables (Customers and Sellers) without a Users table?"	Model Answer: Without a generalized Users superclass, we cannot enforce system-wide uniqueness on login credentials like email, and shared security attributes get duplicated.
Speaker 2: Conceptual EER Architecture & Disjoint Specialization (d)	Screen: Tab 1: Interactive EER Architecture Canvas (Top half: Users & Circle 'd') Time: 1.5 mins
Spoken Script: <i>"Thank you, Speaker 1. Looking at our interactive Chen EER Canvas on screen, you will see how we architected NovaCart conceptually.</i> <i>At the top, we have our Superclass Entity: Users. Notice the rectangle in indigo. The superclass generalizes all common attributes: user_id as the Primary Key, full_name, email, and account creation timestamp.</i> <i>Now, notice the purple circle labeled 'd' right below it. In formal DBMS theory, this circle represents Disjoint Specialization.</i> <i>Why is the 'd' constraint so powerful? Because it defines exclusive set membership: an entity in Users can belong to at most one subclass—it can be a Customer XOR a Seller, but never both simultaneously in our model. Furthermore, participation is total: every registered person must have a defined role.</i> <i>By introducing this disjoint specialization circle, we have mathematically eliminated sparse NULLs at the conceptual level! Next, Speaker 3 will walk you through how our specialized subclasses and their 1:N relationships operate."</i>	
DBMS Keywords to Emphasize: Superclass Users, Chen notation (Rectangle/Ovals), Disjoint Circle (d), Exclusive membership (Customer XOR Seller), Zero NULLs.	Handover Segue: "Speaker 3 will now explain the specialized subclasses and their 1:N relationships."
Examiner Question: "What is the difference between disjoint specialization ('d') and overlapping specialization ('o')?"	Model Answer: Disjoint 'd' means exclusive subsets—a user is either a buyer or a seller. Overlapping 'o' would allow one person to simultaneously hold both roles under a single identity.

Speaker 3: Subclasses & 1:N Relationship Ratios	Screen: Tab 1: Interactive EER Canvas (Middle section: Customers, Sellers, Diamonds) Time: 1.5 mins
Spoken Script: "Thank you, Speaker 2. Directly below the disjoint node, you can see our two specialized Subclasses: Customers on the left and Sellers on the right, shown in crimson. Each subclass holds only the attributes unique to that persona. Customers holds <code>shipping_address</code>, <code>phone_number</code>, and <code>loyalty_tier</code>. Sellers holds <code>gst_number</code>, <code>shop_name</code>, <code>rating</code>, and <code>business_category</code>. Now, let's trace the relationships represented by the Chen diamonds: 1. On the left, we have the diamond labeled 'places'. This represents a 1:N binary relationship between Customers and Orders. One customer can place many orders, but every order belongs to exactly one customer. 2. On the right, we have the diamond labeled 'offers'. This is another 1:N relationship between Sellers and Products. One merchant offers multiple catalog products, but every product is strictly owned by one merchant. Both strong entities—Orders and Products—stand at the heart of our commercial engine. But how do orders and products interact with each other? That brings us to our biggest modeling challenge, which Speaker 4 will now address."	
DBMS Keywords to Emphasize: Subclasses Customers & Sellers, Chen diamonds (places, offers), 1:N Cardinality ratio, Total vs Partial participation.	Handover Segue: "Passing to Speaker 4 to explain how we resolved the Many-to-Many relationship."
Examiner Question: "Why does Orders have total participation with Customers?"	Model Answer: Because an order cannot exist in a vacuum; every order tuple must be placed by a legitimate registered customer.
Speaker 4: The M:N Challenge & Associative Bridge Entity	Screen: Tab 1: Interactive EER Canvas (Bottom section: green diamond 'contains' & Order_Items) Time: 1.5 mins
Spoken Script: "Thank you, Speaker 3. In any real shopping cart, one order can contain multiple different products, and one product can appear in thousands of customer orders. This is a classic Many-to-Many (M:N) Relationship, marked on our canvas by the green diamond labeled 'contains'. In relational database theory, you cannot represent an M:N relationship directly inside either parent table. If you try to store comma-separated product IDs inside an order row, you create multi-valued attribute chaos that makes indexing impossible and breaks SQL joins. Our solution is the standard ER-to-relational modeling technique: we decompose the M:N relationship into an Associative Entity—also known as a Bridge Table—called <code>Order_Items</code>. Notice what <code>Order_Items</code> does: it breaks one complex M:N relationship into two clean 1:N relationships: Orders to <code>Order_Items</code> is 1:N, and Products to <code>Order_Items</code> is 1:N. Crucially, <code>Order_Items</code> also carries relationship attributes: the exact quantity purchased, the <code>unit_price</code> at the moment of checkout, and the line subtotal. Now, let's interact with this canvas in real time. Speaker 5 will demonstrate our live Element Inspector."	
DBMS Keywords to Emphasize: Many-to-Many (M:N) ratio, Multi-valued attributes, Associative entity / Bridge table (<code>Order_Items</code>), Relationship attributes (<code>qty</code> , <code>price</code> , <code>subtotal</code>).	Handover Segue: "Now Speaker 5 will take control of the live prototype to inspect these elements."
Examiner Question: "Why do we store unit_price in Order_Items if Products already has price?"	Model Answer: Because catalog prices fluctuate over time. Storing historical <code>unit_price</code> in <code>Order_Items</code> preserves auditability so past order totals remain permanently accurate.

Speaker 5: Interactive Architectural Inspector (Tab 1 Live Demo)	Screen: Tab 1: Element Inspector Dropdown & Live SQLite Tuples Preview Time: 1.5 mins
Spoken Script: "Thank you, Speaker 4. Let's move from theoretical notation to live software verification. Below our vector canvas, we built an Interactive Conceptual Element Inspector. As I click this dropdown, we can select any entity or relationship in the schema. Let's select Customers (Subclass Entity): • The inspector immediately pulls up its formal classification: Subclass Entity Set via EER Specialization. • Notice its Primary Key: <code>user_id</code>. And its Foreign Key: <code>user_id</code> referencing <code>Users(user_id)</code> with <code>ON DELETE CASCADE</code>. • And on the right column, the prototype executes a live <code>SELECT * FROM Customers LIMIT 5</code>; query directly against our SQLite database. Next, let's switch the dropdown to Order_Items (Associative / Bridge Entity): • Here, you clearly see how it resolves the M:N relationship, carrying foreign keys for both <code>order_id</code> and <code>product_id</code>. • The live data preview displays the exact instances stored on disk right now. Everything you saw on the conceptual Chen diagram is backed by a physical relational engine. But how did we translate these ER concepts into physical tables? Speaker 6 will explain our exact schema mapping rules."	
DBMS Keywords to Emphasize: Live element inspection, Entity metadata, Primary/Foreign keys, Real-time SQL query preview, SQLite disk storage.	Handover Segue: "Over to Speaker 6 to explain our ER-to-Relational Mapping Rules."
Examiner Question: "Where is this database running?"	Model Answer: It is running locally in SQLite 3 with Write-Ahead Logging (WAL) enabled, and all queries are executed dynamically against <code>novacart_dbms.db</code> .

Speaker 6: ER-to-Relational Mapping & Key Inheritance Rules

Screen: Tab 2: EER Specialization & Inheritance Engine (Top cards: Disjointness & Key Rules) | Time: 1.5 mins

Spoken Script:

"Thank you, Speaker 5. Translating conceptual EER diagrams into SQL tables requires strict adherence to **ER-to-Relational Mapping Rules**.

In our architecture, we implemented 4 core mapping rules:

- 1. **Rule 1: Specialization Mapping (PK = FK Inheritance):** In both Customers and Sellers, the column user_id is defined as the **PRIMARY KEY**, but it is simultaneously a **FOREIGN KEY** referencing Users(user_id) with ON DELETE CASCADE. This guarantees that no customer or seller can exist without a parent user tuple, and deleting a user automatically cleans up their subclass record with zero orphan rows.
- 2. **Rule 2: 1:N Relationship Mapping:** For places and offers, we embedded the primary key of the '1' side as a foreign key on the 'N' side (customer_id in Orders, seller_id in Products).
- 3. **Rule 3: M:N Decomposition Mapping:** Order_Items maps the M:N relationship with two foreign keys and an auto-incrementing surrogate key.
- 4. **Rule 4: Domain & Integrity Enforcement:** We enforce domain rules like CHECK(stock_quantity >= 0) and role IN ('Customer', 'Seller') directly in DDL.

Now, let's see key inheritance in action. **Speaker 7** will demonstrate live entity reconstruction and register a new user live."

DBMS Keywords to Emphasize: ER-to-relational mapping, Subclass PK=FK inheritance, ON DELETE CASCADE, Orphan row prevention, Domain CHECK constraints.

Handover Segue: "Speaker 7 will now demonstrate live entity reconstruction and insertion."

Examiner Question: "Why use user_id as PK in Customers instead of an auto-increment customer_id?"

Model Answer: Using user_id as both PK and FK creates an unshakeable 1:1 relationship between the superclass and subclass, enforces disjointness, and avoids redundant surrogate key indexes.

Speaker 7: Live Entity Instantiation & Dynamic Query Reconstruction

Screen: Tab 2: Dynamic Entity Reconstruction & New Specialized Entity Form | Time: 2.0 mins

Spoken Script: "Thank you, Speaker 6. Now let's test our inheritance engine live on screen. Under Live Dynamic Entity Reconstruction, I can pick any user in our system. For example, let's select User #2: Marcus Vance [Seller]. - Instantly, the system executes an EER Natural Equi-Join Query: <code>SELECT u.*, s.* FROM Users u JOIN Sellers s ON u.user_id = s.user_id WHERE u.user_id = 2;</code> - Notice how Marcus's identity attributes (name, email) from Users are seamlessly combined with his merchant attributes (shop name, GST number, rating) from Sellers! Now, let's do something exciting: let's instantiate a brand-new entity live in front of you! - I'll select the radio button for 'Customer'. - Let's enter Full Name: 'Priya Sharma', and Email: 'priya.sharma@domain.edu'. - For Customer attributes, let's add shipping address, phone number, and select 'Gold' tier. - Now, I click 'Insert Entity across Relational Hierarchy'. Watch what happened: in a single atomic action, SQLite inserted Priya into Users, grabbed the generated user_id, and immediately inserted her shipping and loyalty details into Customers using that exact same user_id! Now that we have live customers and merchants, let's see how they trade. Speaker 8 will take us to the live marketplace."	
DBMS Keywords to Emphasize: Natural equi-join, Dynamic reconstruction, Live tuple instantiation, Atomic multi-table insertion, Subclass registration.	Handover Segue: "Speaker 8 will now demonstrate our live Marketplace & Transaction Flow."
Examiner Question: "What happens if the second INSERT fails during user creation?"	Model Answer: Both operations run inside an atomic transaction block. If the subclass insertion fails, the entire transaction rolls back so no orphan user row is left in the superclass table.

Speaker 8: Live Marketplace & Transactional Data Flow (Tab 3 Demo)	Screen: Tab 3: Live Marketplace & Transaction Flow (Configuration & Preview cards) Time: 1.5 mins
Spoken Script: "Thank you, Speaker 7. We are now on Tab 3: Live Marketplace & Transaction Flow. Here, our conceptual entities come alive in a commercial marketplace: • In Step 1 on the left, we configure an order. In the Customer dropdown, you see registered customers from our Customers subclass table. • In the Product dropdown, you see catalog items owned by verified merchants from our Sellers subclass table. Let's select Customer #1: Elena Rostova (Platinum Tier). • For the product, let's select Product #1: NovaBook Pro 15 (\$1,299.99). Notice that current stock is clearly displayed. • Let's set purchase quantity to 2 units. Now look at the right card: the system computes the exact gross total: \$2,599.98. When I click this purple button labeled 'Execute Transactional Checkout', it will trigger an industrial multi-table transaction spanning Orders, Order_Items, and Products. To explain exactly what happens under the hood during this click, I hand over to Speaker 9."	
DBMS Keywords to Emphasize: Customer persona, Merchant product catalog, Computed transaction amount, Real-time ordering.	Handover Segue: "Speaker 9 will now walk you through the transaction execution under the hood."
Examiner Question: "Can a customer buy more items than the stock quantity?"	Model Answer: No, the UI caps quantity at current inventory, and the database schema enforces CHECK(stock_quantity >= 0) so any negative stock attempt is immediately aborted.

Speaker 9: ACID Transaction Timeline & Database Consistency

Screen: Tab 3: Transaction Execution Timeline & Product Catalog Cards | Time: 1.5 mins

Spoken Script:

"Thank you, Speaker 8. Let's click the checkout button and watch the live execution timeline!"

Look at the step-by-step transaction timeline that just rendered:

- 1. **Step 1: BEGIN IMMEDIATE TRANSACTION;** SQLite acquires an exclusive write lock under Write-Ahead Logging (WAL) mode to guarantee isolation.
- 2. **Step 2: INSERT INTO Orders;** A new order header record is created, linking customer_id via Foreign Key.
- 3. **Step 3: INSERT INTO Order_Items;** The Many-to-Many bridge entity materializes! It records order_id, product_id, quantity, unit price, and subtotal.
- 4. **Step 4: UPDATE Products;** Inventory is decremented by exactly 2 units. Here, SQLite verifies our integrity constraint: CHECK(stock_quantity >= 0).
- 5. **Step 5: COMMIT;** All three table modifications are permanently flushed to disk.

This guarantees full **ACID compliance**:

- **Atomicity:** All three writes succeed together or none at all.
- **Consistency:** Stock cannot fall below zero.
- **Isolation:** No dirty reads from concurrent queries.
- **Durability:** Changes survive any unexpected system shutdown.

Now, how do we query this complete transaction history across all 6 tables? **Speaker 10** will show you our Multi-Table Join Graph."

DBMS Keywords to Emphasize: BEGIN TRANSACTION, Exclusive write lock, WAL mode, M:N bridge materialization, CHECK(stock_quantity >= 0), COMMIT, ACID properties.

Handover Segue: "Speaker 10 will now show how we traverse this data using our Multi-Table Join Graph."

Examiner Question: "What happens if power fails between inserting the order and updating the stock?"

Model Answer: Because of SQLite WAL mode and atomicity, uncommitted transactions in the write-ahead log are automatically rolled back upon recovery; dirty partial writes never occur.

Speaker 10: Multi-Table Join Graph & Traversal Engine (Tab 4 Demo)	Screen: Tab 4: Multi-Table Join Graph & Traversal Explorer Time: 1.5 mins
Spoken Script: "Thank you, Speaker 9. Welcome to Tab 4: Multi-Table Join Graph & Traversal Explorer. When we decomposed our database into separate entities—Users, Customers, Sellers, Orders, Products, and Order_Items—we ensured zero NULLs and clean referential integrity. But in the real world, management wants a single, unified view of sales. This screen proves that our ER-to-relational design can be re-assembled losslessly. Notice the pipeline banner: Users → Customers → Orders → Order_Items → Products → Sellers Look at the interactive table below: in a single SQL execution, our query traverses all six relational tables using natural foreign key equi-joins: • We display the Order ID, the Customer's Name, their Loyalty Tier, the Product Purchased, the Quantity, the Unit Price, the Subtotal, the Merchant Shop Name, and the Delivery Status. Notice: Zero duplicate rows, Zero loss of data, and Zero sparse NULL columns. Every column is populated with complete contextual integrity. To present our physical schema constraints and conclude our defense, I pass to our final presenter, Speaker 11."	
DBMS Keywords to Emphasize: 6-Table join traversal, Lossless relational reconstruction, Natural equi-join pipeline, Foreign key chaining, Zero NULLs.	Handover Segue: "Speaker 11 will now conclude our presentation with the physical constraints inspector and viva defense."
Examiner Question: "Is a 6-table join slow in relational databases?"	Model Answer: Not when foreign keys are indexed. In our schema, joins occur strictly on indexed primary keys and foreign key integers (user_id, order_id, product_id), which execute in logarithmic time $O(\log N)$.

Speaker 11: Schema Inspector, Viva Defense & Grand Conclusion	Screen: Tab 5: Relational Schema & Constraints Inspector, then sidebar PDF download Time: 2.0 mins
Spoken Script: "Thank you, Speaker 10. To conclude our evaluation, we arrive at Tab 5: Relational Schema & Constraints Inspector. Here, examiners can inspect the raw physical SQLite DDL statements that govern our platform. When I select Products, you can see the active FOREIGN KEY (seller_id) REFERENCES Sellers(user_id) ON DELETE RESTRICT. Why ON DELETE RESTRICT here instead of CASCADE? Because if a vendor decides to leave our platform, the database must protect financial auditability—it strictly prevents deleting a seller if historical orders or catalog items reference them. In contrast, selecting Customers shows ON DELETE CASCADE: deleting a user account cleanly cascades down to remove their customer profile, preventing orphaned identity records. To summarize our presentation: 1. We proved that EER Disjoint Specialization ('d') eliminates sparse NULL proliferation. 2. We demonstrated that Associative Bridge Entities (Order_Items) cleanly decompose complex M:N relationships into 1:N relations while preserving relationship attributes. 3. We proved that ER-to-Relational Mapping Rules enforce bulletproof referential integrity with CASCADE and RESTRICT constraints. 4. And we demonstrated live that multi-table transactions maintain complete ACID consistency. We have also compiled our complete architecture, mathematical diagrams, and viva answers into a downloadable Workflow PDF Report right here in the sidebar for your review. On behalf of all 11 members of Team NovaCart, thank you for your time, and we now welcome any questions from the panel!"	
DBMS Keywords to Emphasize: ON DELETE RESTRICT vs ON DELETE CASCADE, Financial auditability, Prevention of orphan records, End-to-end ER/EER synthesis, Publication-quality report.	Handover Segue: "Smile, invite panel questions, and keep the Streamlit app ready to navigate to any tab if an examiner requests live verification."
Examiner Question: "What is the single biggest architectural achievement of NovaCart?"	Model Answer: Demonstrating how conceptual EER modeling (disjoint specialization and M:N decomposition) directly maps to a high-performance physical SQLite database with zero NULLs and guaranteed ACID transactional integrity.

3. Stage & Team Coordination Protocol

Rule & Role	Execution Protocol for Team Success
The Designated Navigator:	Assign one teammate (e.g., Speaker 1 or 5) to control the mouse and keyboard smoothly without looking down. When a speaker begins their section, the navigator switches to that tab proactively.
Passing the Mic:	Always end your speech with the exact handover line: 'I now pass to Speaker X to demonstrate...' This ensures no awkward pauses or silence.
Handling Examiner Interruptions:	If a professor cuts in during your speech, stay calm! Use the exact answer provided in the 'Examiner Question & Model Answer' box on your card. Do not guess.
Backup Database Reset:	If someone accidentally enters invalid test data or buys out all stock during practice, click '■ Reset Baseline Database ' in the sidebar to instantly restore pristine demo tuples.